

Jazyk SQL

Co je SQL

- **SQL = Structured Query Language**
- česky: strukturovaný dotazovací jazyk
- slouží ke komunikaci s relačními databázemi
- používá se pro:
 - vytváření databázových objektů
 - vkládání dat
 - čtení dat
 - úpravu dat
 - mazání dat
 - správu oprávnění
 - řízení transakcí

SQL se používá například v databázích:

- PostgreSQL
- MySQL
- Microsoft SQL Server
- Oracle Database
- SQLite

Charakteristika SQL

- SQL je **deklarativní jazyk**
- říkáme, **co** chceme získat
- nepopisujeme přesně, **jak** to má databáze udělat
- databázový systém sám rozhodne, jak dotaz nejlépe vykoná

Příklad:

```
SELECT *  
FROM Studenti  
WHERE Vek >= 18;
```

Význam:

- chceme všechny studenty, kterým je alespoň 18 let
- neřešíme, jak databáze data fyzicky najde

Historie SQL

- SQL vzniklo ve firmě **IBM**
- původně se jmenovalo **SEQUEL**
- později se název zkrátil na **SQL**
- první standard SQL vznikl v roce 1986
- SQL je standardizovaný jazyk, ale jednotlivé databázové systémy mají vlastní odchylky

SQL dialekty

- různé databázové systémy používají vlastní rozšíření SQL
- základní principy jsou podobné
- některé funkce nebo syntaxe se ale mohou lišit

Příklady:

- T-SQL — Microsoft SQL Server
- PL/SQL — Oracle
- PL/pgSQL — procedurální jazyk v PostgreSQL
- MySQL SQL — varianta SQL v MySQL

Poznámka:

- běžné `SELECT` , `INSERT` , `UPDATE` , `DELETE` jsou podobné napříč databázemi
- rozdíly bývají hlavně u funkcí, datových typů, procedur a speciálních možností

Rozdělení SQL příkazů

SQL příkazy se často dělí do několika skupin:

- DDL
- DML
- DQL
- DCL
- TCL

DDL

Co je DDL

- **DDL = Data Definition Language**
- jazyk pro definici dat
- pracuje se strukturou databáze
- mění databázové objekty, například tabulky

DDL příkazy:

- CREATE
 - ALTER
 - DROP
 - TRUNCATE
-

CREATE

- slouží k vytvoření databázového objektu
- například:
 - tabulky
 - databáze
 - pohledu
 - indexu

Příklad vytvoření tabulky:

```
CREATE TABLE Studenti (  
    Id INT PRIMARY KEY,  
    Jmeno VARCHAR(50) NOT NULL,  
    Prijmeni VARCHAR(50) NOT NULL,  
    Vek INT  
);
```

ALTER

- slouží k úpravě existujícího databázového objektu
- například přidání, změna nebo odstranění sloupce

Příklad přidání sloupce:

```
ALTER TABLE Studenti  
ADD Email VARCHAR(100);
```

Příklad změny tabulky:

```
ALTER TABLE Studenti  
ALTER COLUMN Email VARCHAR(150);
```

Poznámka:

- přesná syntaxe ALTER se může mezi databázemi lišit

DROP

- smaže databázový objekt
- například tabulku
- odstraní strukturu i data
- používá se opatrně

Příklad:

```
DROP TABLE Studenti;
```

Význam:

- smaže celou tabulku `Studenti`
- smaže i její data

TRUNCATE

- smaže všechna data z tabulky
- struktura tabulky zůstane
- bývá rychlejší než `DELETE`
- obvykle nemaže tabulku samotnou

Příklad:

```
TRUNCATE TABLE Studenti;
```

Rozdíl:

- `DROP` smaže tabulku celou
- `TRUNCATE` smaže jen data v tabulce
- `DELETE` maže řádky a může mít podmínku `WHERE`

DML

Co je DML

- **DML = Data Manipulation Language**
- jazyk pro manipulaci s daty
- pracuje s řádky v tabulkách

DML příkazy:

- `INSERT`
- `UPDATE`

- DELETE

Poznámka:

- Někdy se sem řadí i SELECT , ale často se SELECT vyčleňuje jako DQL.

DQL

Co je DQL

- **DQL = Data Query Language**
- jazyk pro dotazování dat
- hlavním příkazem je SELECT

Příkaz:

- SELECT

DCL

Co je DCL

- **DCL = Data Control Language**
- jazyk pro řízení přístupu
- pracuje s oprávněními uživatelů

DCL příkazy:

- GRANT
- REVOKE

GRANT

- uděluje oprávnění uživateli

Příklad:

```
GRANT SELECT ON Studenti TO uzivatel;
```

Význam:

- uživatel získá právo číst data z tabulky Studenti

REVOKE

- odebírá oprávnění uživateli

Příklad:

```
REVOKE SELECT ON Studenti FROM uživatel;
```

Význam:

- uživateli se odebere právo číst tabulku Studenti

TCL

Co je TCL

- **TCL = Transaction Control Language**
- jazyk pro řízení transakcí
- používá se tam, kde chceme provést více operací jako jeden celek

TCL příkazy:

- COMMIT
- ROLLBACK
- BEGIN TRANSACTION

Transakce

- **transakce** = skupina databázových operací
- buď se provedou všechny operace
- nebo se neprovede žádná
- používá se kvůli konzistenci dat

Příklad:

```
BEGIN TRANSACTION;

UPDATE Ucty
SET Zustatek = Zustatek - 1000
WHERE Id = 1;

UPDATE Ucty
SET Zustatek = Zustatek + 1000
WHERE Id = 2;

COMMIT;
```

Význam:

- peníze se odečtou z jednoho účtu
- připsou se na druhý účet
- změny se potvrdí pomocí COMMIT

COMMIT

- potvrdí transakci
- změny se uloží natrvalo

```
COMMIT;
```

ROLLBACK

- zruší transakci
- vrátí databázi do původního stavu před transakcí

```
ROLLBACK;
```

Databázové objekty

Tabulka

- základní objekt relační databáze
- obsahuje řádky a sloupce
- uchovává data

Příklad:

```
CREATE TABLE Studenti (  
    Id INT PRIMARY KEY,  
    Jmeno VARCHAR(50),  
    Prijmeni VARCHAR(50)  
);
```

Pohled

- **view** = virtuální tabulka
- je založený na SQL dotazu
- neukládá nutně vlastní data
- zobrazuje data z jedné nebo více tabulek
- může zjednodušit práci s databází
- může skrýt citlivé sloupce

Příklad:

```
CREATE VIEW AktivniStudenti AS  
SELECT Jmeno, Prijmeni
```

```
FROM Studenti  
WHERE Aktivni = 1;
```

Index

- datová struktura pro rychlejší vyhledávání
- funguje podobně jako rejstřík v knize
- zrychluje čtení
- může zpomalit zápis, protože index se musí aktualizovat

Příklad:

```
CREATE INDEX idx_student_prijmeni  
ON Studenti(Prijmeni);
```

Sekvence / auto increment

- generuje postupné hodnoty
- často se používá pro primární klíče

Příklad v MySQL:

```
CREATE TABLE Studenti (  
    Id INT AUTO_INCREMENT PRIMARY KEY,  
    Jmeno VARCHAR(50)  
);
```

Příklad v SQL Serveru:

```
CREATE TABLE Studenti (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    Jmeno VARCHAR(50)  
);
```

Uložená procedura

- kus SQL kódu uložený v databázi
- lze ji opakovaně spouštět
- může obsahovat složitější logiku
- syntaxe se liší podle databázového systému

Obecný význam:

- databázová funkce / postup uložený přímo v DB

Integritní omezení

Co jsou integritní omezení

- **integritní omezení** = pravidla pro data v tabulkách
- hlídají správnost a konzistenci dat
- používají se při vkládání a úpravě dat

Hlavní omezení:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT

PRIMARY KEY

- primární klíč
- jednoznačně identifikuje řádek
- nesmí být NULL
- hodnota se nesmí opakovat

Příklad:

```
CREATE TABLE Studenti (  
    Id INT PRIMARY KEY,  
    Jmeno VARCHAR(50)  
);
```

FOREIGN KEY

- cizí klíč
- odkazuje na primární klíč jiné tabulky
- zajišťuje referenční integritu
- brání odkazům na neexistující záznamy

Příklad:

```
CREATE TABLE Tridy (  
    Id INT PRIMARY KEY,  
    Nazev VARCHAR(20)  
);
```

```
CREATE TABLE Studenti (  
    Id INT PRIMARY KEY,  
    Jmeno VARCHAR(50),  
    TridaId INT,  
    FOREIGN KEY (TridaId) REFERENCES Tridy(Id)  
);
```

Význam:

- student může odkazovat jen na existující třídu

UNIQUE

- zajišťuje, že se hodnota ve sloupci neopakuje
- používá se například pro e-mail nebo uživatelské jméno

Příklad:

```
CREATE TABLE Uzivatele (  
    Id INT PRIMARY KEY,  
    Email VARCHAR(100) UNIQUE  
);
```

NOT NULL

- sloupec nesmí být prázdný
- hodnota musí být vyplněná

Příklad:

```
CREATE TABLE Studenti (  
    Id INT PRIMARY KEY,  
    Jmeno VARCHAR(50) NOT NULL  
);
```

CHECK

- hodnota musí splnit podmínku

Příklad:

```
CREATE TABLE Produkty (  
    Id INT PRIMARY KEY,  
    Nazev VARCHAR(100),  
    Cena DECIMAL(10,2) CHECK (Cena > 0)  
);
```

DEFAULT

- nastaví výchozí hodnotu
- použije se, pokud při vložení nezadáme vlastní hodnotu

Příklad:

```
CREATE TABLE Studenti (  
    Id INT PRIMARY KEY,  
    Aktivni BIT DEFAULT 1  
);
```

SELECT

Co je SELECT

- nejpoužívanější SQL příkaz
- slouží ke čtení dat z databáze
- může vybírat:
 - všechny sloupce
 - konkrétní sloupce
 - vypočítané hodnoty
 - agregované hodnoty

Základní SELECT

```
SELECT *  
FROM Studenti;
```

- * znamená všechny sloupce
- vybere všechny řádky z tabulky Studenti

Výběr konkrétních sloupců

```
SELECT Jmeno, Prijmeni  
FROM Studenti;
```

- vybere pouze sloupce Jmeno a Prijmeni

Alias

- alias umožňuje přejmenovat sloupec nebo tabulku ve výsledku
- používá se AS

Příklad:

```
SELECT Jmeno AS KrestniJmeno  
FROM Studenti;
```

Příklad u tabulky:

```
SELECT s.Jmeno  
FROM Studenti AS s;
```

Pořadí klauzulí v SELECT

Typické pořadí zápisu:

```
SELECT sloupce  
FROM tabulka  
JOIN dalsi_tabulka ON podminka  
WHERE podminka_radku  
GROUP BY sloupce  
HAVING podminka_skupin  
ORDER BY sloupce;
```

Význam:

1. SELECT — co chci zobrazit
2. FROM — odkud беру data
3. JOIN — jaké tabulky připojuji
4. WHERE — filtrace řádků
5. GROUP BY — seskupení řádků
6. HAVING — filtrace skupin
7. ORDER BY — řazení výsledků

WHERE

Co je WHERE

- filtruje řádky podle podmínky
- používá se u SELECT , UPDATE i DELETE

Příklad:

```
SELECT *  
FROM Studenti  
WHERE Vek >= 18;
```

Operátory ve WHERE

Operátor	Význam
=	rovná se
<> nebo !=	nerovná se
>	větší než
<	menší než
>=	větší nebo rovno
<=	menší nebo rovno
AND	a zároveň
OR	nebo
NOT	negace
BETWEEN	mezi hodnotami
IN	je v seznamu
LIKE	podobnost textu
IS NULL	je prázdné / null

AND a OR

```
SELECT *  
FROM Studenti  
WHERE Vek >= 18 AND Trida = '4.A';
```

BETWEEN

```
SELECT *  
FROM Produkty  
WHERE Cena BETWEEN 100 AND 500;
```

IN

```
SELECT *  
FROM Studenti  
WHERE Trida IN ('4.A', '4.B');
```

LIKE

```
SELECT *  
FROM Studenti  
WHERE Prijmeni LIKE 'Nov%';
```

Význam:

- najde příjmení začínající na Nov

IS NULL

```
SELECT *  
FROM Studenti  
WHERE Email IS NULL;
```

Pozor:

```
WHERE Email = NULL
```

- je špatně, protože NULL se porovnává pomocí IS NULL .

JOIN

Co je JOIN

- JOIN slouží ke spojení více tabulek
- používá se podle vazeb mezi tabulkami
- typicky se spojuje primární klíč s cizím klíčem

Příklad tabulek:

Studenti

Id	Jmeno	Tridald
1	Jan	1
2	Eva	2

Tridy

Id	Nazev
1	4.A
2	4.B

INNER JOIN

- vrátí jen záznamy, které mají odpovídající záznam v obou tabulkách

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
INNER JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Výsledek:

Jmeno	Nazev
Jan	4.A
Eva	4.B

LEFT JOIN

- vrátí všechny řádky z levé tabulky
- pokud v pravé tabulce není odpovídající záznam, doplní se `NULL`

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
LEFT JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Použití:

- když chceme vypsát všechny studenty, i ty bez třídy

RIGHT JOIN

- vrátí všechny řádky z pravé tabulky
- pokud v levé tabulce není odpovídající záznam, doplní se `NULL`

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
RIGHT JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Poznámka:

- často se dá přepsat pomocí `LEFT JOIN` opačným pořadím tabulek

FULL OUTER JOIN

- vrátí záznamy z obou tabulek
- pokud chybí odpovídající záznam, doplní se `NULL`

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
```

```
FULL OUTER JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Poznámka:

- ne všechny databáze podporují FULL OUTER JOIN

GROUP BY

Co je GROUP BY

- seskupuje řádky podle hodnoty ve sloupci
- používá se s agregačními funkcemi
- například pro počítání, součty nebo průměry

Příklad:

```
SELECT Trida, COUNT(*) AS PocetStudentu  
FROM Studenti  
GROUP BY Trida;
```

Význam:

- spočítá počet studentů v každé třídě

HAVING

Co je HAVING

- filtruje skupiny vytvořené pomocí GROUP BY
- podobá se WHERE
- rozdíl je v tom, že WHERE filtruje řádky před seskupením
- HAVING filtruje skupiny po seskupení

Příklad:

```
SELECT Trida, COUNT(*) AS PocetStudentu  
FROM Studenti  
GROUP BY Trida  
HAVING COUNT(*) > 20;
```

Význam:

- vypíše jen třídy, kde je více než 20 studentů

WHERE vs. HAVING

WHERE	HAVING
filtruje řádky	filtruje skupiny
používá se před GROUP BY	používá se po GROUP BY
nepoužívá agregační výsledek	často používá agregační funkce

Příklad:

```
SELECT Trida, COUNT(*) AS PocetStudentu
FROM Studenti
WHERE Aktivni = 1
GROUP BY Trida
HAVING COUNT(*) > 20;
```

Význam:

- nejdřív vybere aktivní studenty
- pak je seskupí podle třídy
- nakonec ponechá jen skupiny s více než 20 studenty

ORDER BY

Co je ORDER BY

- řadí výsledek dotazu
- ASC = vzestupně
- DESC = sestupně

Příklad:

```
SELECT *
FROM Studenti
ORDER BY Prijmeni ASC;
```

Příklad sestupně:

```
SELECT *
FROM Studenti
ORDER BY Vek DESC;
```

Funkce v SQL

Agregační funkce

- pracují nad více řádky
- často se používají s GROUP BY

Funkce	Význam
COUNT()	počet
SUM()	součet
AVG()	průměr
MIN()	minimum
MAX()	maximum

Příklad:

```
SELECT COUNT(*) AS PocetStudentu
FROM Studenti;
```

```
SELECT AVG(Vek) AS PrumernyVek
FROM Studenti;
```

Textové funkce

- pracují s textem
- liší se podle databázového systému

Příklady:

```
SELECT UPPER(Jmeno)
FROM Studenti;
```

```
SELECT LOWER(Prijmeni)
FROM Studenti;
```

Datumové funkce

- pracují s datem a časem
- syntaxe se liší podle databáze

Příklad:

```
SELECT CURRENT_DATE;
```

nebo:

```
SELECT GETDATE();
```

Poznámka:

- `CURRENT_DATE` je běžné například v PostgreSQL
- `GETDATE()` je typické pro SQL Server

INSERT

Co je `INSERT`

- vkládá nový řádek do tabulky

Příklad:

```
INSERT INTO Studenti (Jmeno, Prijmeni, Vek)
VALUES ('Jan', 'Novák', 18);
```

Vložení více řádků

```
INSERT INTO Studenti (Jmeno, Prijmeni, Vek)
VALUES
('Jan', 'Novák', 18),
('Eva', 'Svobodová', 19),
('Petr', 'Dvořák', 18);
```

UPDATE

Co je `UPDATE`

- upravuje existující data v tabulce
- často se používá s `WHERE`

Příklad:

```
UPDATE Studenti
SET Vek = 19
WHERE Id = 1;
```

Pozor:

```
UPDATE Studenti
SET Vek = 19;
```

- bez `WHERE` se změní věk všem studentům

DELETE

Co je DELETE

- maže řádky z tabulky
- často se používá s WHERE

Příklad:

```
DELETE FROM Studenti  
WHERE Id = 1;
```

Pozor:

```
DELETE FROM Studenti;
```

- bez WHERE smaže všechny řádky v tabulce
- struktura tabulky zůstane

CREATE

Co je CREATE

- vytváří databázové objekty
- patří do DDL

Příklad vytvoření tabulky:

```
CREATE TABLE Produkty (  
  Id INT PRIMARY KEY,  
  Nazev VARCHAR(100) NOT NULL,  
  Cena DECIMAL(10,2) CHECK (Cena > 0)  
);
```

ALTER

Co je ALTER

- mění existující databázový objekt
- nejčastěji tabulku
- patří do DDL

Příklad přidání sloupce:

```
ALTER TABLE Produkty  
ADD Popis VARCHAR(255);
```

Příklad smazání sloupce:

```
ALTER TABLE Produkty  
DROP COLUMN Popis;
```

DROP

Co je DROP

- odstraní databázový objekt
- například tabulku
- smaže strukturu i data
- patří do DDL

Příklad:

```
DROP TABLE Produkty;
```

Praktický příklad dotazu

Zadání

Chceme zjistit počet aktivních studentů v jednotlivých třídách, ale jen pro třídy, kde je více než 20 aktivních studentů.

```
SELECT Tridy.Nazev, COUNT(Studenti.Id) AS PocetStudentu  
FROM Tridy  
INNER JOIN Studenti ON Studenti.TridaId = Tridy.Id  
WHERE Studenti.Aktivni = 1  
GROUP BY Tridy.Nazev  
HAVING COUNT(Studenti.Id) > 20  
ORDER BY PocetStudentu DESC;
```

Vysvětlení:

- SELECT vybere název třídy a počet studentů
- FROM určí hlavní tabulku
- INNER JOIN připojí studenty
- WHERE vybere jen aktivní studenty
- GROUP BY seskupí data podle tříd
- HAVING ponechá jen skupiny s více než 20 studenty

- ORDER BY seřadí výsledek podle počtu studentů sestupně